Lab 4 - Potpourri

Ayden Frost

Introduction

In this lab you'll review and get practice with a variety of concepts, methods, and tools you've encountered thus far.

Part 1 - All about Quarto

Question 1

a. Add each of strings below to the code chunk provided in your document, render the document, and determine if the string is a proper code chunk label. If not, explain why and describe how you could fix it so the document renders.

   - Chunk label 1:

```
#| label: a-label
#| with-a-line-break
```

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

   It doesn't render with a line break so I had to put it all in one line.

   - Chunk label 2:

```
#| label: areaaaaaaaaaaaaaaaaaaaaaaallllllllllllllllllyyyyyloooooooooooooooooonglabel
```

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

   This label works as is

- Chunk label 3:

```
#| label: label with spaces
```

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

The document doesn't render if there are spaces, there has to be dashes instead

- Chunk label 4:

```
#| label: label-with-dashes
```

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

This label works as is

> **Tip**
>
> Try each label option in the code chunk provided in your document. If it gives you an error (document doesn't render), you know it's not a proper chunk label.

b. Which of the chunk label options above is the best option? Explain your reasoning in 1-2 sentences.

The short label with the dashes (chunk label 4) works and is easy to read and remember.

Question 2

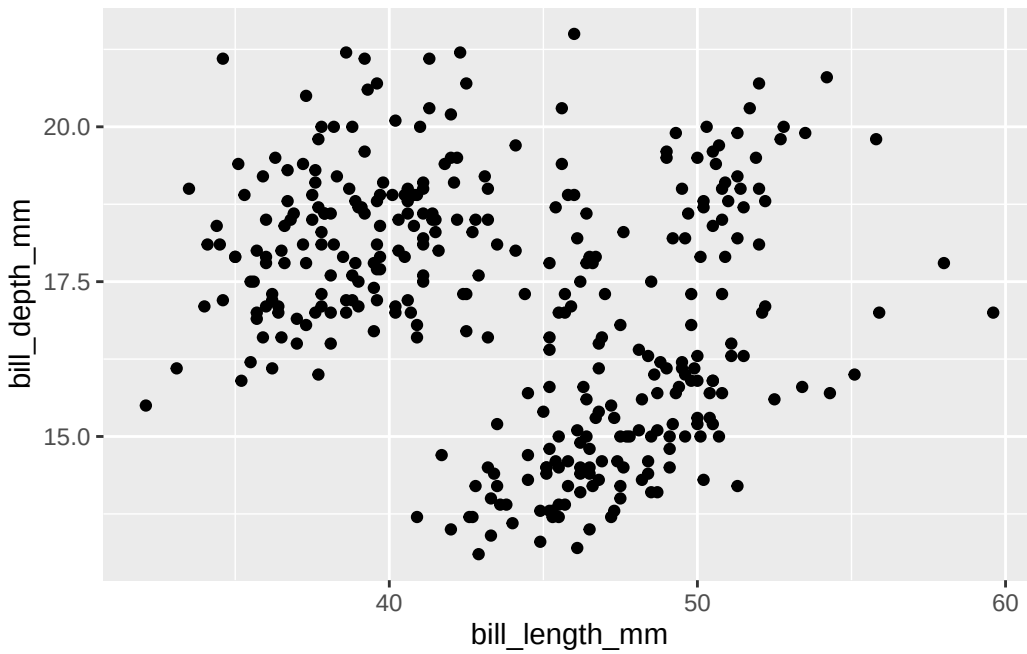
You have the following code chunk:

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

Add the following code chunk options, one at a time, and set each to `false` and then to `true`. After each value, render your document and observe its effect. Ultimately, choose the values that are the most appropriate for this code chunk. Based on the behaviors you observe, describe what each code chunk option does.

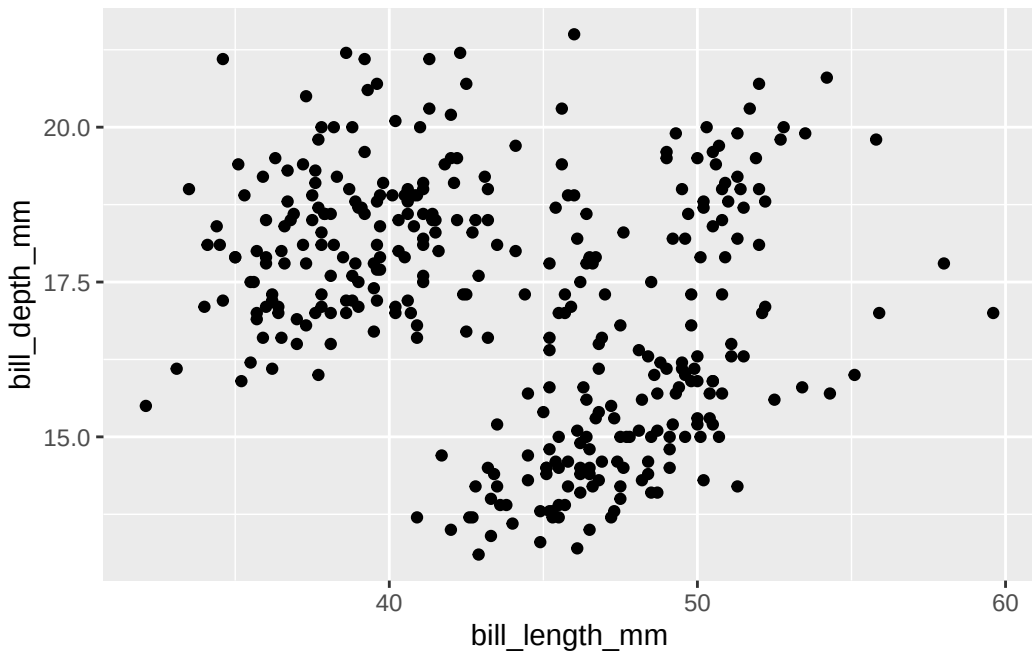
- `echo`
- `warning`
- `eval`

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```

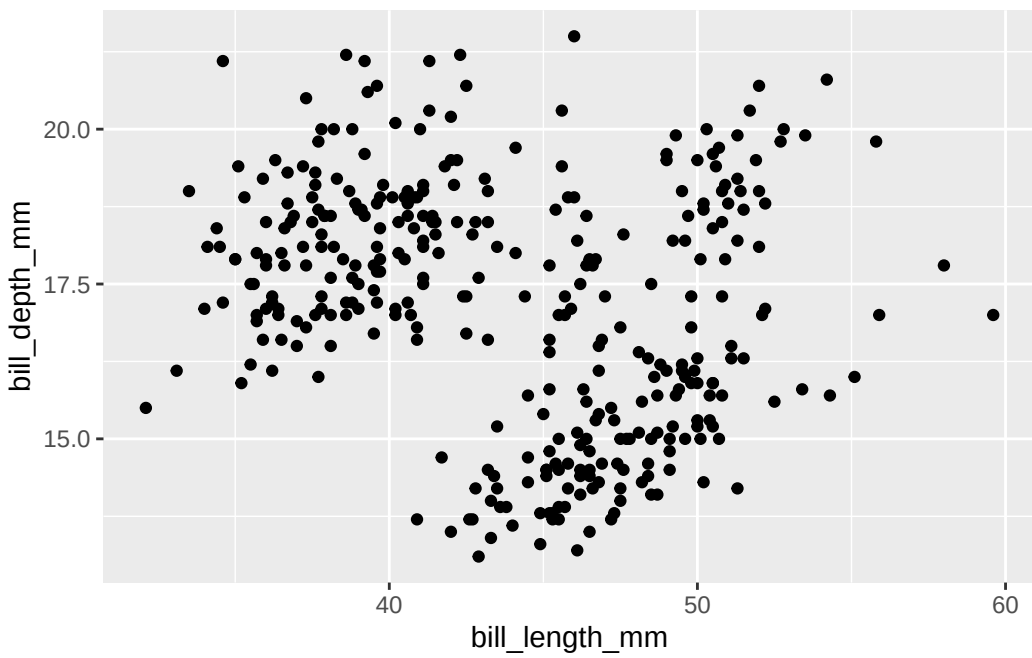



```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```

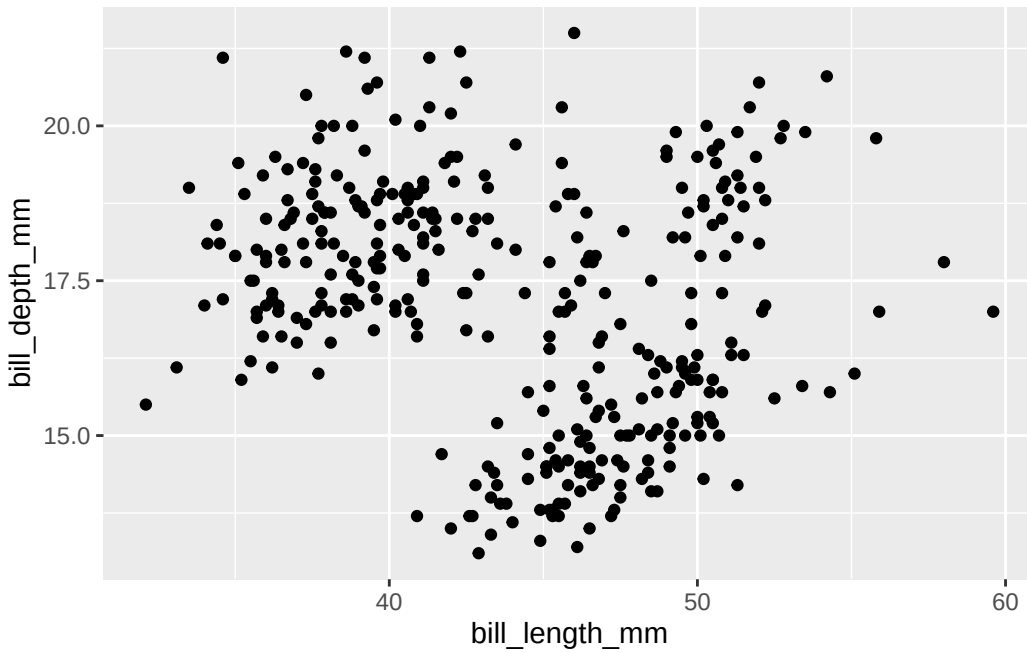



```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```



```r
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

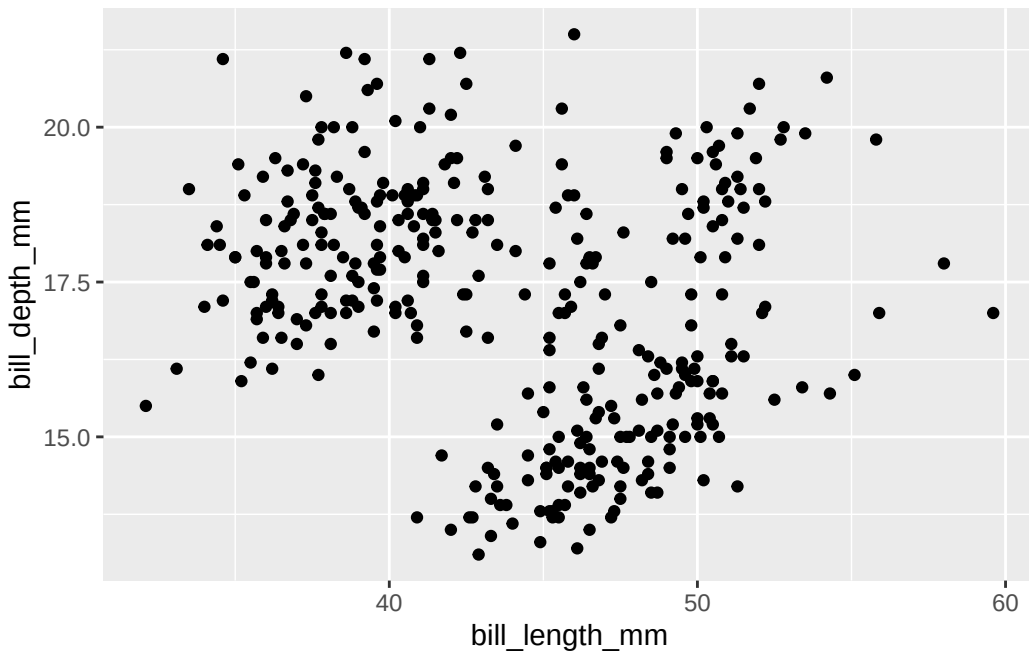
```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```




```r
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

```r
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



I would choose to put echo and eval as true and warning as false. Echo seems to decide if the code shows up in the document, eval seems to decide if the code is run at all, and warning decides if the warnings are shown (if there is one).

Question 3

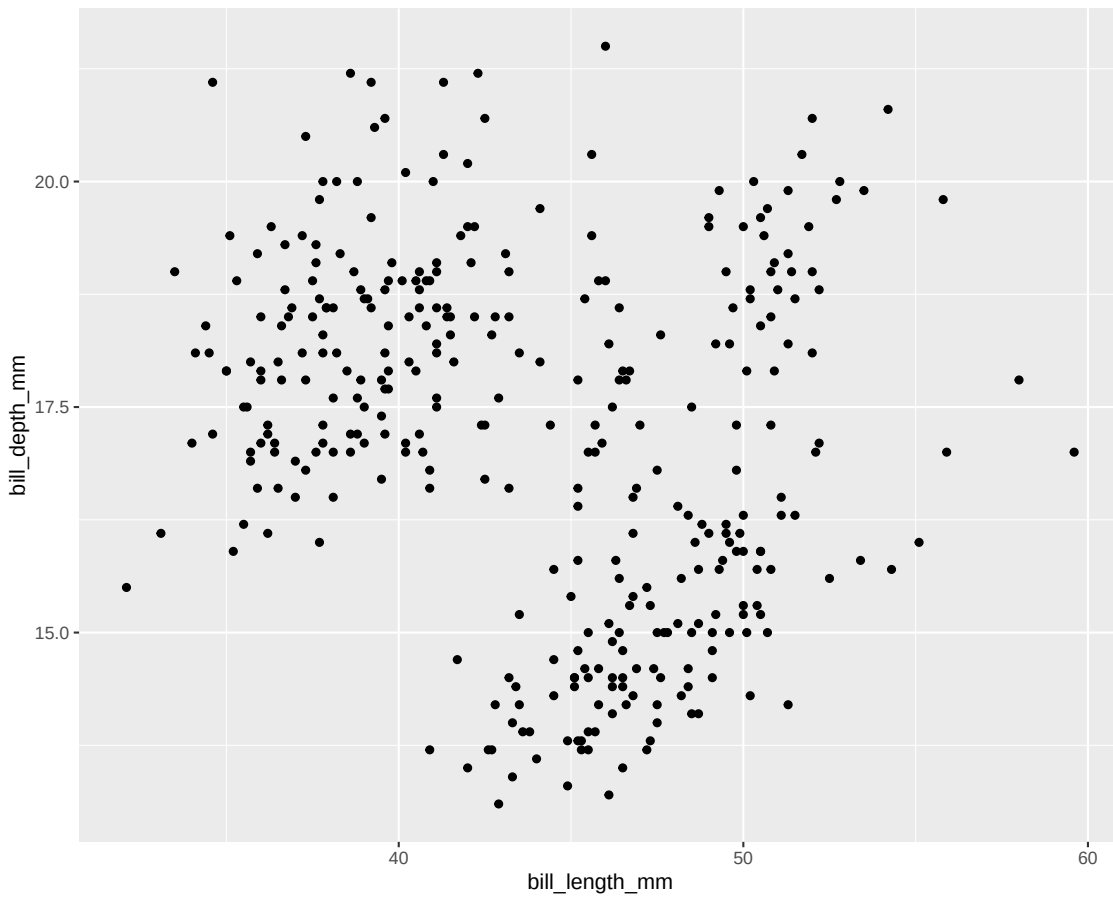
a. You have the following code chunk again.

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

Add `fig-width` and `fig-asp` as code chunk options. Try setting `fig-width` to values between 1 and 10. Try setting `fig-asp` to values between 0.1 and 1. Re-render the document after each value and observe its effect. Ultimately, choose values that make the plot look visually pleasing in the rendered document. Based on the behavior you observe, describe what each chunk option does.

```
ggplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

```
Warning: Removed 2 rows containing missing values or values outside the scale range
(`geom_point()`).
```



I chose 8 for fig-width and 0.8 for fig-asp. fig-width controls how wide the plot is and fig-asp controls how tall it is but as a function of width.

b. You have the following code chunk, but look carefully, it's not exactly the same!

```
gplot(penguins, aes(x = bill_length_mm, y = bill_depth_mm)) +
  geom_point()
```

Add `error` as a code chunk option and set it to `false` and then set it to `true`. After each value, render your document and observe its effect. Ultimately, choose the value that allows you to render your document without altering the code. Based on the behavior you observe, describe what this code chunk option does.

I had to use error: true to allow it to render the document because the error chunk option controls if the the document can be rendered even when there is an error in the code

> **Tip**
>
> Reading the documentation might also be hepful.

Render, commit, and push one last time. Make sure that you commit and push all changed documents and your Git pane is completely empty before proceeding.

Part 2 - Misrepresentation

Question 4

The following chart was shared by @GraphCrimes on X/Twitter on September 3, 2022.

a. What is misleading about this graph?

   The percentages are not represented properly, especially for energy and royal mail where 14-15% takes up half the bar.

b. Suppose you wanted to recreate this plot, with improvements to avoid its misleading pitfalls from part (a). You would obviously need the data from the survey in order to be able to do that. How many observations would this data have? How many variables (at least) should it have, and what should those variables be?

   It would have 1858 observations, I think the minimum number of variables would have to be 2, response and service.

c. Load the data for this survey from `data/survation.csv`. Confirm that the data match the percentages from the visualization. That is, calculate the percentages of public sector, private sector, don't know for each of the services and check that they match the percentages from the plot.

```r
read.csv(survation.csv)
```

```
Error:
! object 'survation.csv' not found
```

I can't find the csv file so I can't do this question

Question 5

Create an improved version of the visualization. Your improved visualization:

- should also be a stacked bar chart with services on the y-axis, presented in the same order as the original plot, and services to create the segments of the plot, and presented in the same order as the original plot
- should have the same legend location
- should have the same title and caption
- does not need to have a bolded title or a gray background

How does the improved visualization look different than the original? Does it send a different message at a first glance?

I still can't do this question without the csv

> **Tip**
>
> Use `\n` to add a line break to your title. And note that since the title is very long, it might run off the page in your code. That's ok!
> Additionally, the colors used in the plot are `gray`, `#FF3205`, and `#006697`.

Render, commit, and push one last time. Make sure that you commit and push all changed documents and your Git pane is completely empty before proceeding.

Part 3 - DatasauRus

The data frame you will be working with in this part is called `datasaurus_dozen` and it's in the **datasauRus** package. This single data frame contains 13 datasets, designed to show us why data visualization is important and how summary statistics alone can be misleading. The different datasets are marked by the `dataset` variable, as shown in **?@fig-datasaurus**.

> **Note**
>
> If it's confusing that the data frame is called `datasaurus_dozen` when it contains 13 datasets, you're not alone! Have you heard of a baker's dozen?

Here is a peek at the top 10 rows of the dataset:

```
datasaurus_dozen
```

```
# A tibble: 1,846 x 3
   dataset     x     y
   <chr>   <dbl> <dbl>
 1 dino     55.4  97.2
 2 dino     51.5  96.0
 3 dino     46.2  94.5
 4 dino     42.8  91.4
 5 dino     40.8  88.3
 6 dino     38.7  84.9
 7 dino     35.6  79.9
 8 dino     33.1  77.6
 9 dino     29.0  74.5
10 dino     26.2  71.4
# i 1,836 more rows
```

Question 6

In a single pipeline, calculate the mean of `x`, mean of `y`, standard deviation of `x`, standard deviation of `y`, and the correlation between `x` and `y` for each level of the `dataset` variable. Then, in 1-2 sentences, comment on how these summary statistics compare across groups (datasets).

```r
datasaurus_dozen|>
  group_by(dataset)|>
  summarise(
      mean_x = mean(x),
    mean_y = mean(y),
    sd_x = sd(x),
    sd_y = sd(y),
    correlation = cor(x, y)
  )|>
  print(n=13)
```

```
# A tibble: 13 x 6
   dataset    mean_x mean_y  sd_x  sd_y correlation
   <chr>       <dbl>  <dbl> <dbl> <dbl>       <dbl>
 1 away         54.3   47.8  16.8  26.9     -0.0641
 2 bullseye     54.3   47.8  16.8  26.9     -0.0686
 3 circle       54.3   47.8  16.8  26.9     -0.0683
 4 dino         54.3   47.8  16.8  26.9     -0.0645
```

```
 5 dots         54.3   47.8  16.8  26.9     -0.0603
 6 h_lines      54.3   47.8  16.8  26.9     -0.0617
 7 high_lines   54.3   47.8  16.8  26.9     -0.0685
 8 slant_down   54.3   47.8  16.8  26.9     -0.0690
 9 slant_up     54.3   47.8  16.8  26.9     -0.0686
10 star         54.3   47.8  16.8  26.9     -0.0630
11 v_lines      54.3   47.8  16.8  26.9     -0.0694
12 wide_lines   54.3   47.8  16.8  26.9     -0.0666
13 x_shape      54.3   47.8  16.8  26.9     -0.0656
```

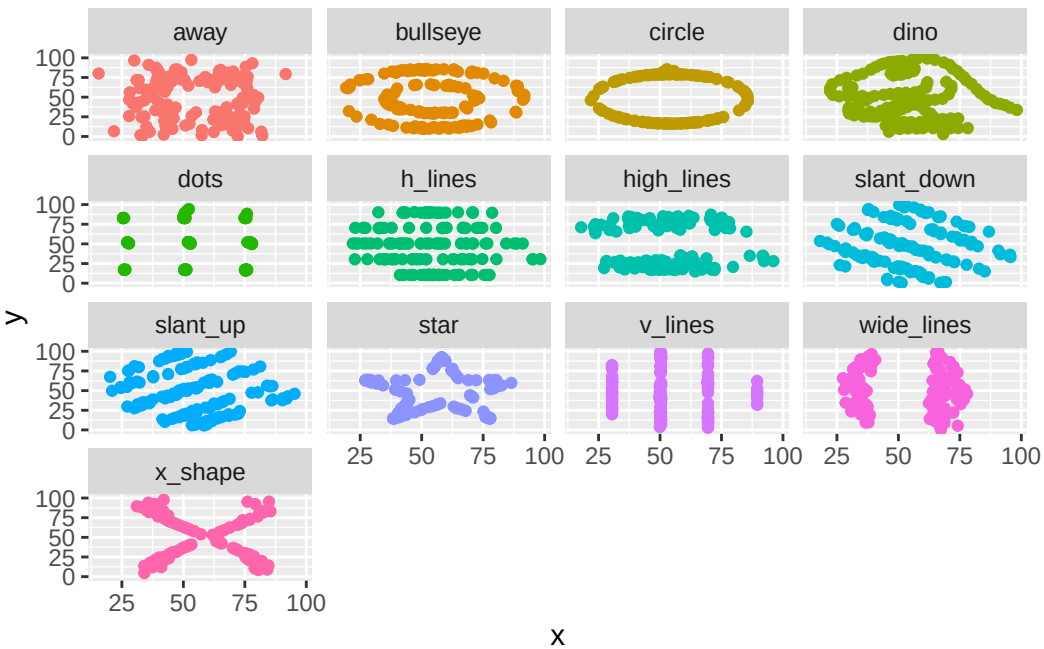
They are the same or very similar values across the data sets, only the correlation values vary.

> **Tip**
>
> There are 13 groups but `tibble`s only print out 10 rows by default. Add `print(n = 13)` as the last step of your pipeline to display all rows.

Question 7

Create a scatterplot of `y` versus `x` and color and facet it by `dataset`. Then, in 1-2 sentences, how these plots compare across groups (datasets). How does your response in this question compare to your response to the previous question and what does this say about using visualizations and summary statistics when getting to know a dataset?

```r
ggplot(datasaurus_dozen, aes(x = x, y = y, color = dataset)) +
  geom_point(show.legend = FALSE) +
  facet_wrap(~ dataset)
```



The plots look very different while having the same statistics, it shows that visually checking data is just as important as looking at the statistics when becoming familiar with a dataset.

> **Tip**
>
> When you both color and facet by the same variable, you'll end up with a redundant legend. Turn off the legend by adding `show.legend = FALSE` to the geom creating the legend.

Render, commit, and push one last time. Make sure that you commit and push all changed documents and your Git pane is completely empty before proceeding.

Grading

The lab is graded out of a total of 28 points.

You can earn up to 4 points on each question:

- 4: Response shows excellent understanding and addresses all or almost all of the rubric items.
- 3: Response shows good understanding and addresses most of the rubric items.
- 2: Response shows understanding and addresses a majority of the rubric items.

- 1: Response shows effort and misses many of the rubric items.
- 0: Response does not show sufficient effort or understanding and/or is largely incomplete.